

Sensen Games

Documentação Técnica e Arquitetural

Softex-Bolsa Futuro Digital(Turma-04-C1-RJ)

22 de janeiro de 2026

Resumo

Este documento apresenta a especificação técnica e arquitetural da plataforma Sensen Games, uma aplicação web desenvolvida no modelo Single Page Application (SPA), voltada à comercialização de jogos digitais independentes. O documento descreve requisitos funcionais e não funcionais, arquitetura de software, gestão de estado, acessibilidade, segurança e estratégias de evolução do sistema.

Conteúdo

1	Introdução	2
2	Requisitos do Sistema	2
2.1	Requisitos Funcionais (RF)	2
2.2	Requisitos Não Funcionais (RNF)	3
3	Arquitetura Tecnológica	3
4	Estrutura de Diretórios e Modularização	4
5	Arquitetura e Diagrama de Componentes	5
5.1	Descrição dos Componentes Arquiteturais	5
6	Gestão de Estado e Dados	5
6.1	StoreContext (E-commerce)	5
6.2	SettingsContext (Configurações)	6
7	Camada de Interface (UI) e Componentes	6
7.1	Sistema de Animação (Reveal)	6
7.2	HeroCarousel	6
7.3	Componentes de Formulário	6
8	Integração com Backend (Mocking)	6
9	Experiência do Utilizador (UX)	7
10	Segurança e Boas Práticas	7

11 Acessibilidade	7
12 Estratégia de Testes	7
13 Escalabilidade e Evolução	7
14 Considerações Finais	7

1 Introdução

A plataforma **Sensen Games** foi concebida como uma solução web responsiva projetada para a descoberta, visualização detalhada e simulação de compra de jogos eletrónicos. O sistema foi projetado com foco em performance, proporcionando uma experiência de utilizador (UX) fluida através de animações de entrada e uma arquitetura modular que permite fácil manutenção e escalabilidade.

2 Requisitos do Sistema

Nesta secção, detalham-se os requisitos identificados e implementados no sistema, divididos entre funcionais (o que o sistema faz) e não funcionais (como o sistema se comporta).

2.1 Requisitos Funcionais (RF)

Os seguintes requisitos definem as funcionalidades centrais disponíveis para o utilizador final:

- **[RF01] Visualização de Catálogo:** O sistema permite ao utilizador visualizar uma lista paginada ou em grade de todos os jogos disponíveis.
- **[RF02] Detalhes do Produto:** O utilizador pode aceder a uma página ou modal com informações detalhadas de um jogo (descrição, preço, avaliação, requisitos de sistema).
- **[RF03] Gestão do Carrinho:** O sistema permite a adição de jogos ao carrinho, remoção de itens e atualização de quantidades.
- **[RF04] Cálculo de Total:** O cálculo do subtotal e do valor total da compra é realizado automaticamente com base nos itens do carrinho.
- **[RF05] Lista de Desejos (Wishlist):** O utilizador pode marcar e desmarcar jogos como favoritos para visualização futura.
- **[RF06] Contacto Institucional:** O sistema fornece um formulário para envio de mensagens diretas à equipa de suporte.
- **[RF07] Newsletter:** O utilizador pode inscrever o seu e-mail para receber novidades (Mailing List).
- **[RF08] Configurações Dinâmicas:** O sistema carrega informações institucionais (redes sociais, logótipos) a partir de uma configuração centralizada, sem necessidade de alteração no código das páginas.

2.2 Requisitos Não Funcionais (RNF)

Estes requisitos foram estabelecidos para garantir a qualidade e respeitar as restrições técnicas do projeto:

- **[RNF01] Responsividade:** A interface adapta-se fluidamente a dispositivos móveis, tablets e desktops (Mobile-First).
- **[RNF02] Single Page Application (SPA):** A navegação entre páginas não recarrega o navegador, garantindo transições instantâneas.
- **[RNF03] Performance:** O tempo de carregamento inicial é minimizado utilizando estratégias de *bundling* otimizadas (Vite).
- **[RNF04] Feedback de Interface:** Todas as ações de criação, atualização ou erro geram notificações visuais (Toasts) para o utilizador.
- **[RNF05] Manutenibilidade:** O código foi desenvolvido em TypeScript com tipagem estrita para reduzir erros em tempo de execução.
- **[RNF06] Estética Visual:** O sistema segue rigorosamente o tema escuro (*Dark Mode*) com a paleta de cores violeta e ardósia definida no Design System.

3 Arquitetura Tecnológica

A solução utiliza uma stack moderna baseada no ecossistema JavaScript/TypeScript, priorizando tipagem estática e compilação rápida.

- **Runtime e Linguagem:** Node.js, TypeScript (Strict Mode).
- **Core Framework:** React 19.
- **Build System:** Vite v7 (foco em ESMódulos nativos).
- **Estilização:** Tailwind CSS v4 (integração via PostCSS e variáveis CSS nativas).
- **Roteamento:** React Router Dom v7.
- **Comunicação HTTP:** Axios (configurado com interceptadores base).
- **Gestão de Formulários:** React Hook Form.
- **Interface e Acessibilidade:** Radix UI (Primitivos Headless) e Lucide React (Ícones).

4 Estrutura de Diretórios e Modularização

A organização do código fonte em `src/` segue uma arquitetura baseada em domínios funcionais. A estrutura abaixo é apresentada de forma condensada para facilitar a leitura e compreensão da hierarquia.

```
src/
|-- api/                # Cliente HTTP
|   |-- axios.ts        # Configuracao singleton do Axios
|-- assets/             # Recursos Estaticos
|   |-- icons/          # Logotipos e svgs
|   |-- styles/         # CSS Global (Tailwind)
|-- components/         # Biblioteca de UI
|   |-- figma/          # Componentes visuais (ImageWithFallback)
|   |-- Forms/          # Formularios (Contact, MailingList)
|   |-- Layout/         # Header, Footer, Layout Wrapper
|   |-- ui/             # Primitivos (Dialog, Icons, Utils)
|   |-- GameModal.tsx   # Modal de visualizacao rapida
|   |-- HeroCarousel.tsx # Slider da Home
|   |-- Reveal.tsx      # Animacao de Scroll
|-- contexts/           # Estado Global
|   |-- SettingsContext.tsx
|   |-- StoreContext.tsx
|   |-- useStore.ts
|-- mocks/              # Dados Simulados
|   |-- gameMock.ts
|   |-- settingsMock.ts
|-- pages/              # Rotas da Aplicacao
|   |-- Contact/        # Pagina de Contato
|   |-- CartPage.tsx    # Carrinho
|   |-- CatalogPage.tsx # Listagem de Jogos
|   |-- GamePage.tsx    # Detalhes do Jogo
|   |-- HomePage.tsx    # Pagina Inicial
|   |-- WishlistPage.tsx
|-- routes/             # Configuracao de Rotas
|   |-- index.tsx
|-- types/              # Definicoes TypeScript
|   |-- Game.ts
|   |-- index.ts
|-- utils/              # Funcoes Auxiliares
|   |-- media.ts
|-- App.tsx             # Componente Raiz
|-- main.tsx            # Entrypoint
```

Árvore de Diretórios Principal

Figura 1: Estrutura de Arquivos do Frontend

5 Arquitetura e Diagrama de Componentes

A aplicação adota uma **Arquitetura Baseada em Componentes** dentro do padrão SPA (Single Page Application). O diagrama abaixo detalha como os componentes de alto nível interagem, evidenciando o fluxo unidirecional de dados e a injeção de dependências via Context API.

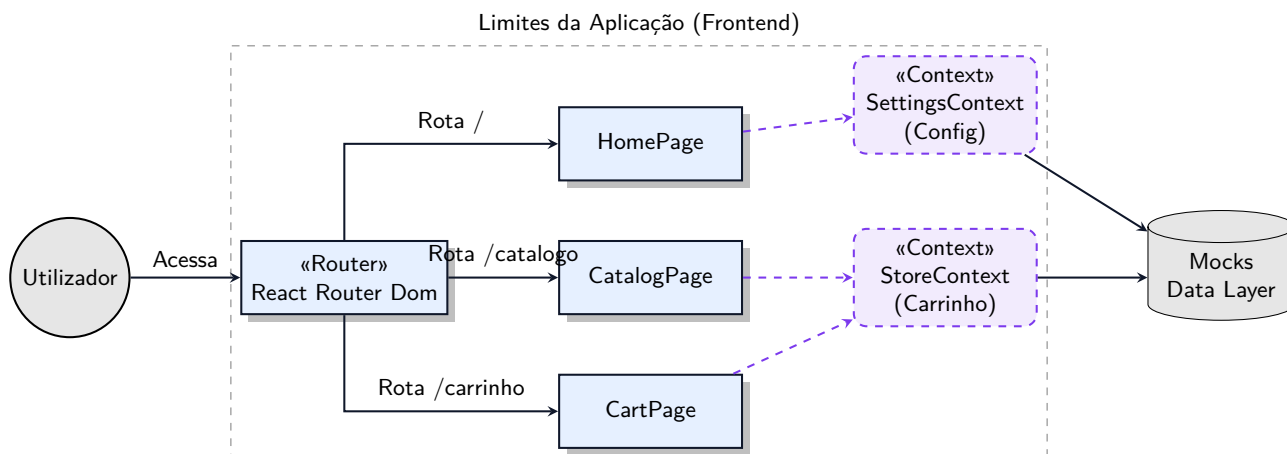


Figura 2: Diagrama de Componentes UML - Nível 2

5.1 Descrição dos Componentes Arquiteturais

1. **Router (Controlador):** Atua como ponto de entrada para decidir qual *Page Controller* instanciar com base na URL.
2. **Page Controllers:** Componentes de alto nível (Views) que orquestram o layout. O estado de negócio não é gerido diretamente aqui, sendo essa responsabilidade delegada aos Contextos.
3. **Context API (Model/State):**
 - *StoreContext:* Implementado como um *Singleton* para a sessão do utilizador, mantendo o estado do carrinho persistente entre navegações.
 - *SettingsContext:* Provê configurações estáticas que são injetadas na inicialização.
4. **Data Layer (Mocks):** Simula-se uma API REST, retornando objetos tipados conforme as interfaces definidas em `src/types`.

6 Gestão de Estado e Dados

A aplicação utiliza a Context API do React para evitar *prop-drilling* e centralizar a lógica de negócios em dois domínios principais.

6.1 StoreContext (E-commerce)

Responsável por toda a lógica transacional do carrinho de compras.

- **Entidade:** Define-se a entidade `CartItem` (estendendo a interface `Game` com `quantity`).
- **Normalização:** Implementa-se a função `getGameKey` para tratar IDs que podem ser numéricos ou strings (slugs), garantindo consistência na manipulação do array.

- Ações Expostas:

- `addToCart(game)`: Lógica de "upsert" (inserção ou incremento).
- `removeFromCart(id)`: Remoção física do item.
- `updateQuantity(id, delta)`: Controlo granular de quantidade.
- `getCartTotal()`: Reducer para cálculo financeiro.

6.2 SettingsContext (Configurações)

As configurações globais da aplicação são centralizadas, desacoplando o conteúdo do código.

- **Propósito:** O contexto fornece dados institucionais (Título do site, Logos, Links de Redes Sociais, Endereços) para componentes como Header, Footer e ContactPage.
- **Fonte de Dados:** Os dados são alimentados através de `src/mocks/settingsMock.ts`.
- **Benefício Arquitetural:** Permite a alteração de dados como telefone ou link do Instagram num único arquivo, refletindo em toda a aplicação instantaneamente.

7 Camada de Interface (UI) e Componentes

7.1 Sistema de Animação (Reveal)

O componente `Reveal.tsx` atua como um wrapper universal para animações de entrada. São utilizados observadores de interseção ou transições CSS para aplicar efeitos de *fade-in* e *slide-up* quando os elementos entram na viewport.

7.2 HeroCarousel

Localizado na `HomePage`, este componente é implementado como um slider rotativo automatizado. A lista de jogos destacados é consumida e são renderizados banners de alta resolução com chamadas para ação (CTA).

7.3 Componentes de Formulário

Os formulários (`ContactForm`, `MailingListForm`) implementam validação robusta no cliente.

- **Tecnologia:** React Hook Form.
- **Feedback:** Integração com `Sonner` para toasts de sucesso/erro e substituição visual do formulário após envio bem-sucedido.

8 Integração com Backend (Mocking)

Como a aplicação opera em modo *frontend-first*, a camada de dados é simulada de forma estruturada.

- **Axios Instance:** O arquivo `src/api/axios.ts` exporta uma instância configurada do cliente HTTP.
- **Mocks:** Os arquivos em `src/mocks/` (`gameMock.ts`, `settingsMock.ts`) seguem rigorosamente as interfaces definidas em `src/types/`.

9 Experiência do Utilizador (UX)

A interface foi desenvolvida com foco em usabilidade, consistência visual e fluidez de interações, utilizando componentes reutilizáveis para garantir uma experiência coesa em toda a aplicação.

10 Segurança e Boas Práticas

O sistema adota práticas preventivas de segurança, incluindo validação rigorosa de dados no *client-side*, e a arquitetura foi planejada para futura implementação de autenticação segura.

11 Acessibilidade

A aplicação segue as recomendações da WCAG 2.1, garantindo navegação eficiente por teclado, uso correto de rótulos ARIA e contraste de cores adequado para utilizadores com baixa visão.

12 Estratégia de Testes

A arquitetura foi desenhada para suportar testes em múltiplos níveis: testes unitários para funções de lógica, testes de componentes para elementos de UI e testes *end-to-end* para fluxos críticos de utilizador.

13 Escalabilidade e Evolução

A estrutura modular do projeto permite a evolução incremental de funcionalidades e a integração futura com um backend real sem a necessidade de refatorações significativas na base de código existente.

14 Considerações Finais

Conclui-se que a plataforma Sensen Games apresenta maturidade técnica e viabilidade como produto digital escalável, combinando tecnologias modernas com práticas sólidas de engenharia de software.